

Base de Datos Distribuidas

SEMANA 13

Heider Sanchez
hsanchez@utec.edu.pe

5.

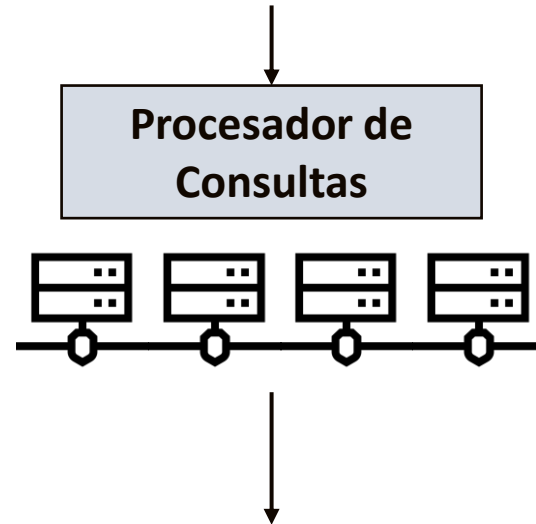


Base de Datos Distribuidas

*Procesamiento de
Consultas Distribuidas*

Procesamiento de Consultas Distribuidas

Consulta de usuario – alto nivel



Comandos de manipulación de datos – bajo nivel

Procesamiento de Consultas Distribuidas

Pasos Generales:

1. Descomposición

- Uno o más árboles de consulta algebraica sobre las relaciones.

2. Localización

- Reemplazo de relaciones por sus fragmentos correspondientes.

3. Optimización

- Reducir la transmisión de tuplas por la red.

Procesamiento de Consultas Distribuidas

Descomposición

- Uno o más árboles de consulta algebraica sobre las relaciones.



Localización

- Reemplazo de relaciones por sus fragmentos correspondientes.



Optimización

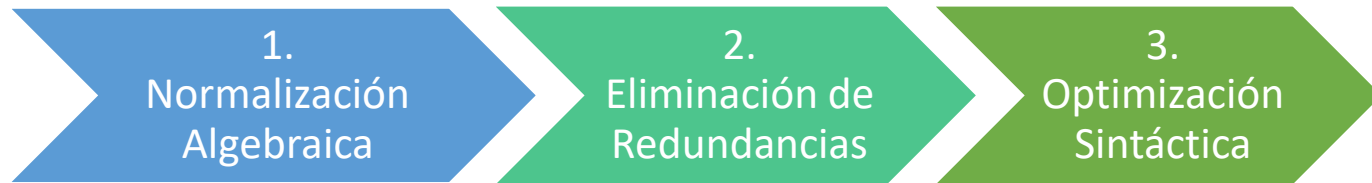
- Reducir la transmisión de tuplas por la red.

Descomposición

UTEC

Descomposición

Pasos:



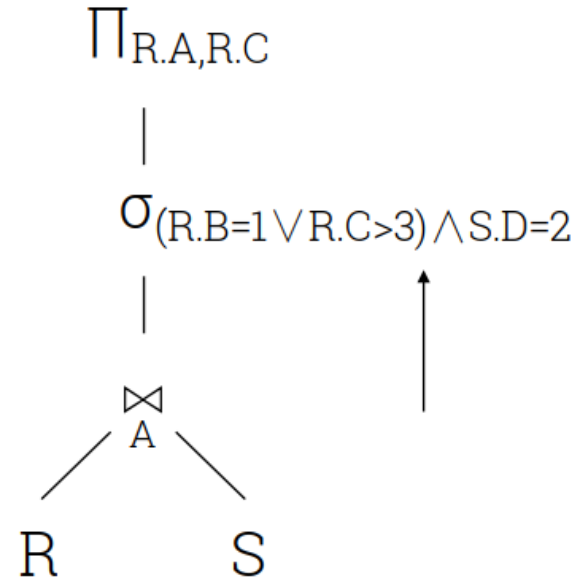
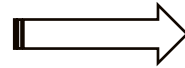
Descomposición

- **Normalización Algebraica**

- Convertir del notación SQL a una forma estándar

Ejemplo:

```
select R.A, R.C
from R, S
where R.A = S.A and
      ((R.B = 1 and S.D = 2) or
       (R.C > 3 and S.D = 2))
```



También se debe detectar las expresiones inválidas

Descomposición

- **Eliminación de Redundancias**

- Por ejemplo: condiciones falsas, condiciones redundantes:

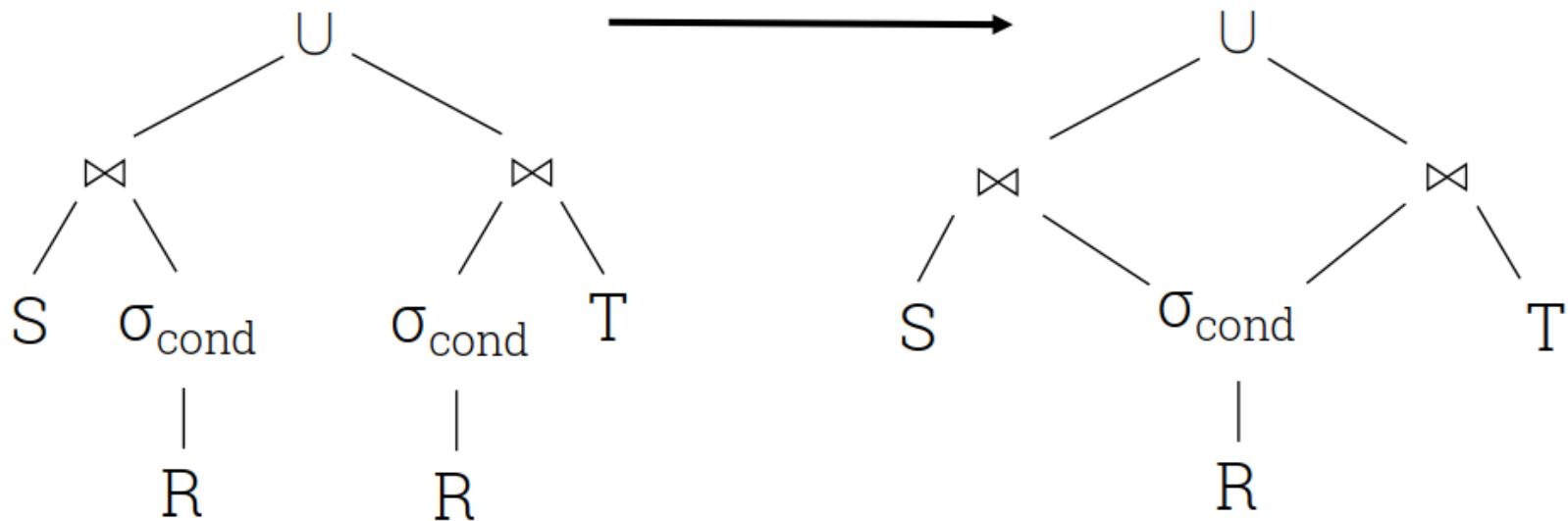
$$(S.A = 1) \wedge (S.A > 5) \Rightarrow \text{false}$$

$$(S.A < 10) \wedge (S.A < 5) \Rightarrow S.A < 5$$

Descomposición

- **Eliminación de Redundancias**

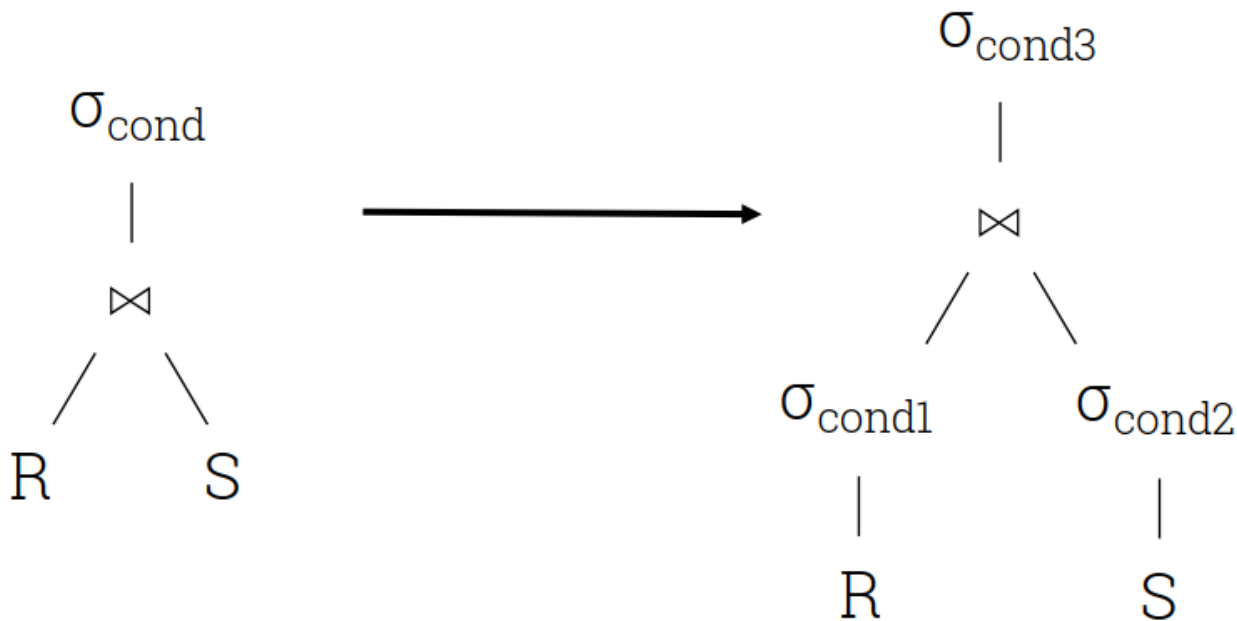
- Por ejemplo, cuando comparten sub-expresiones:



Descomposición

- **Optimización Sintáctica**

- Llevar las condiciones y proyecciones lo más abajo posible.

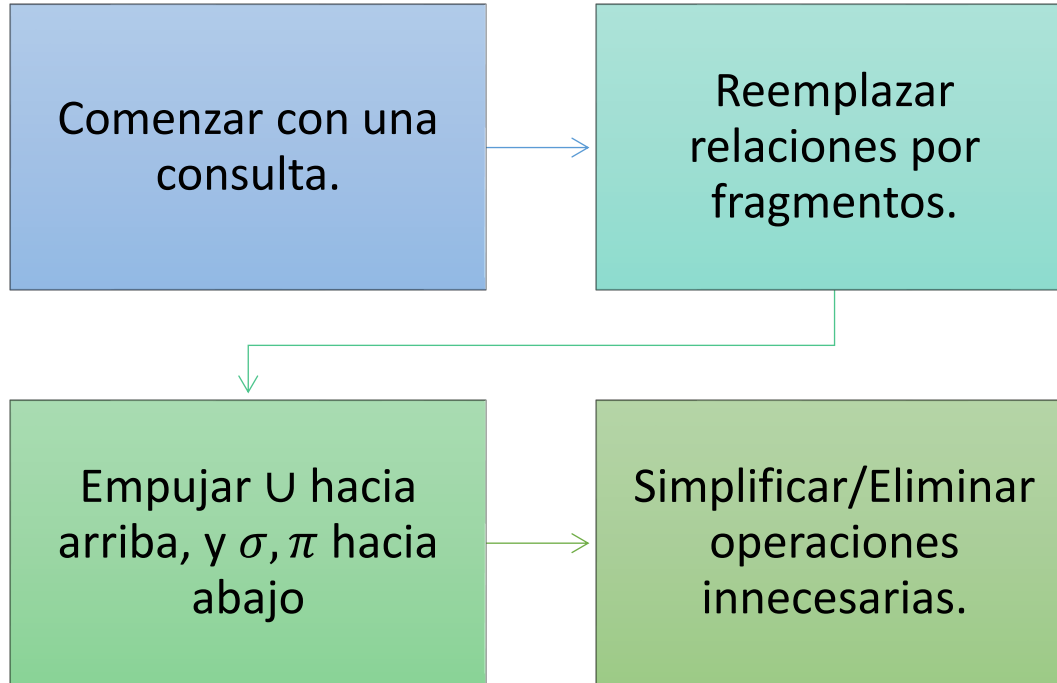


Localización

UTEC

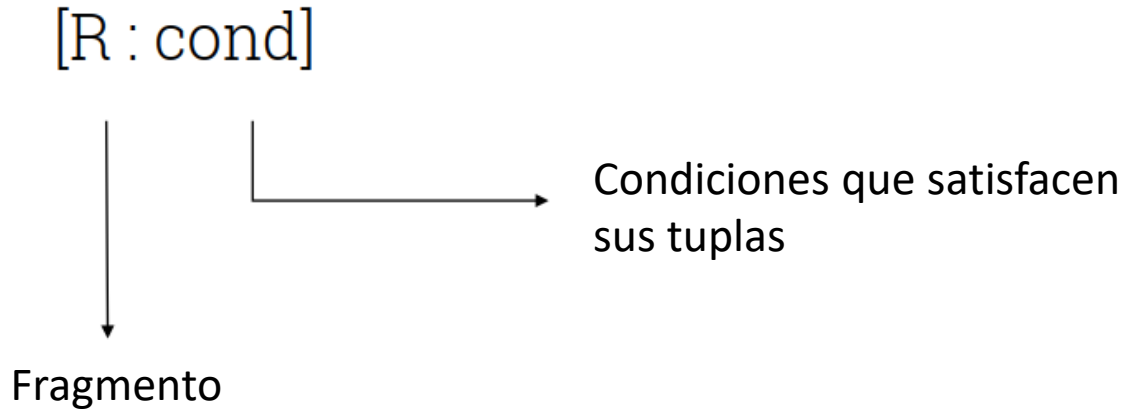
Localización

- Pasos:



Localización

- **Notación para el fragmento:**



Localización

- **Ejemplo 1:**

- Sea la consulta:

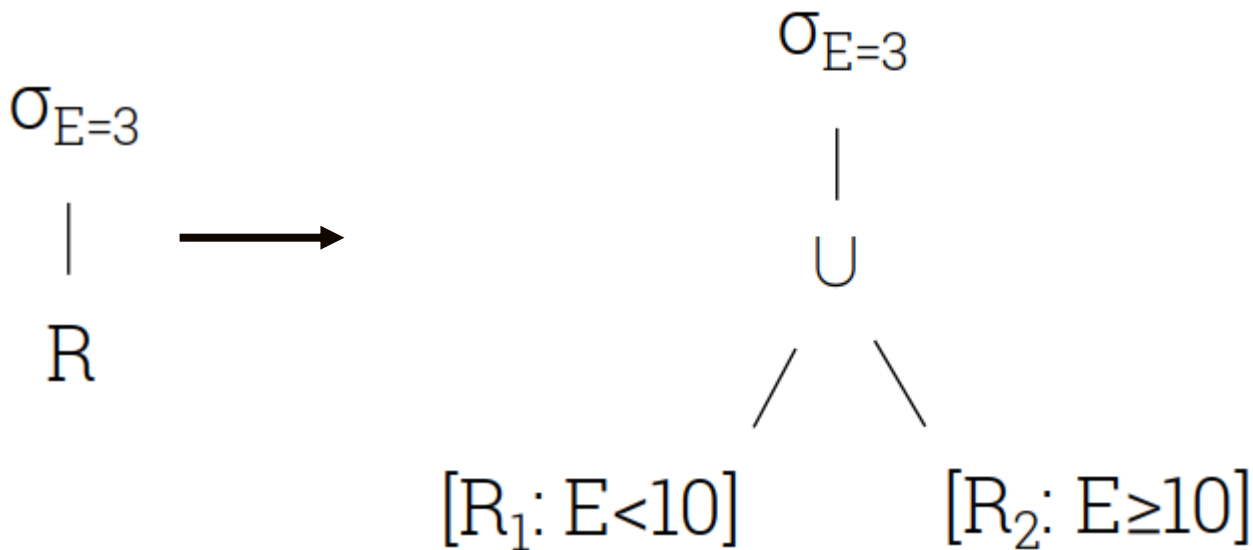
`SELECT * FROM R WHERE R.E = 3`

Y R esta compuesto por los fragmentos

`{ [R1:E <10] , [R2: E >= 10]}`

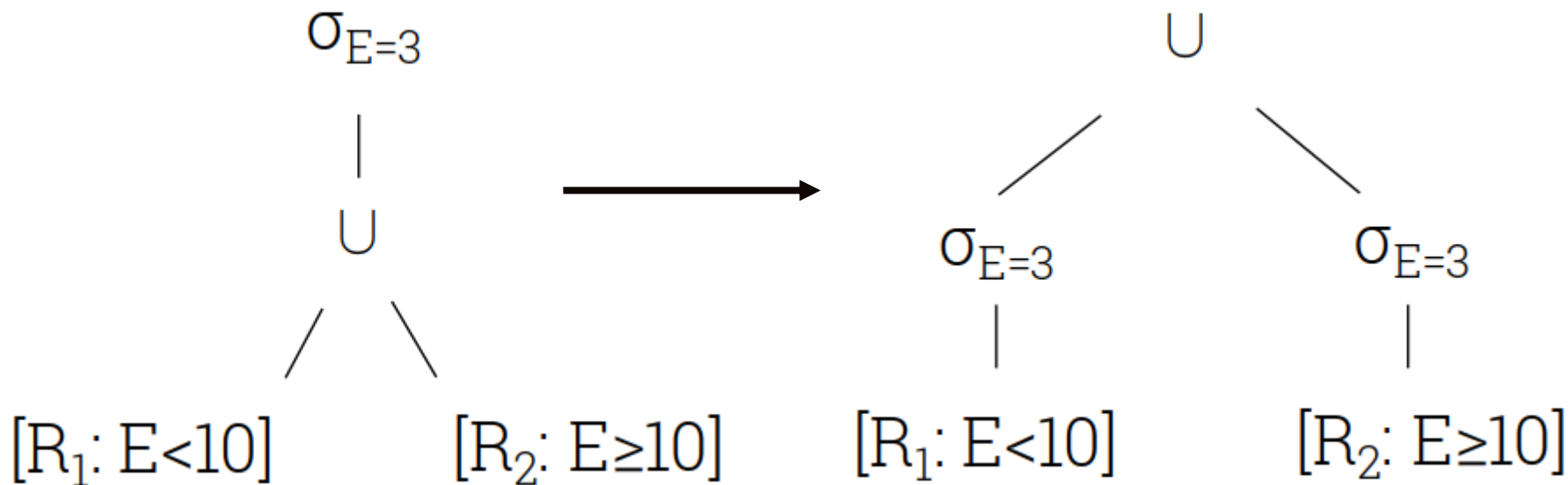
Localización

- **Ejemplo 1:**



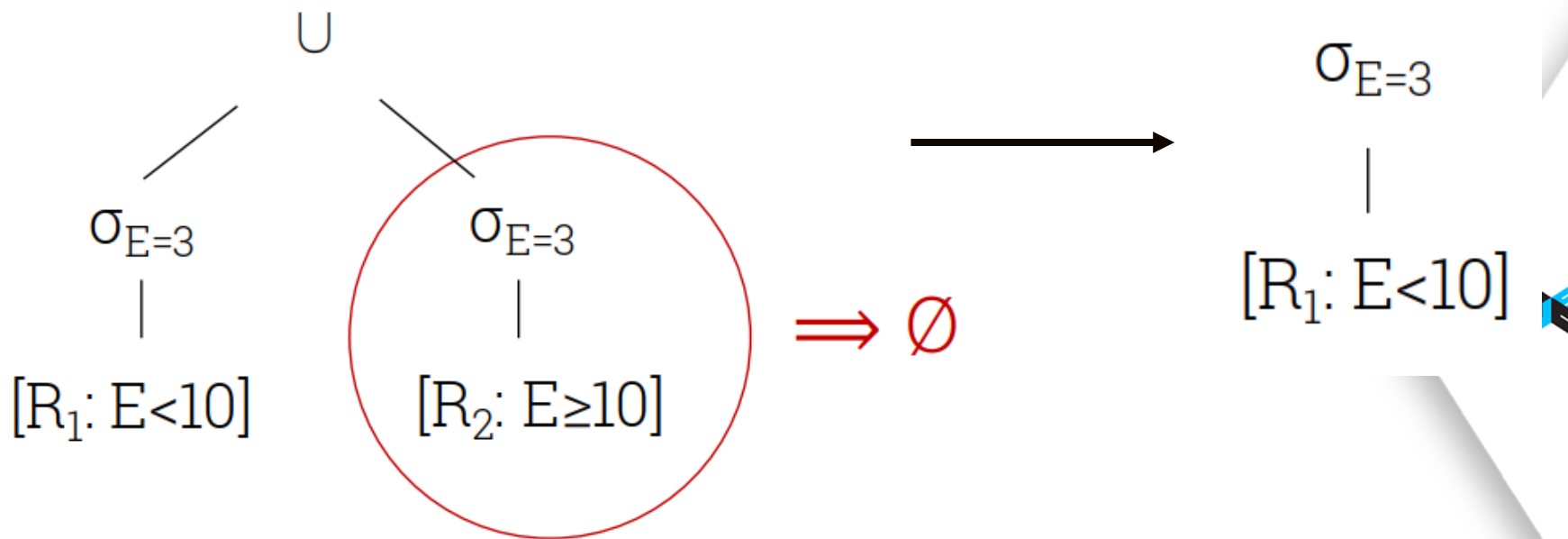
Localización

- **Ejemplo 1:**



Localización

- **Ejemplo 1:**



Localización

- **Regla 1:**

$$\sigma_{c1}[R: c2] \Rightarrow \sigma_{c1}[R: c1 \wedge c2]$$

$$[R: \text{false}] \Rightarrow \emptyset$$

Localización

- **Ejemplo 1:**

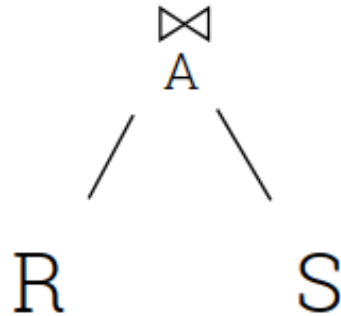
$$\begin{aligned}\sigma_{E=3} [R_2: E \geq 10] &\Rightarrow \sigma_{E=3} [R_2: E=3 \wedge E \geq 10] \\ &\Rightarrow \sigma_{E=3} [R_2: \text{false}] \\ &\Rightarrow \emptyset\end{aligned}$$

Localización

- **Ejemplo 2:**

Sea la consulta:

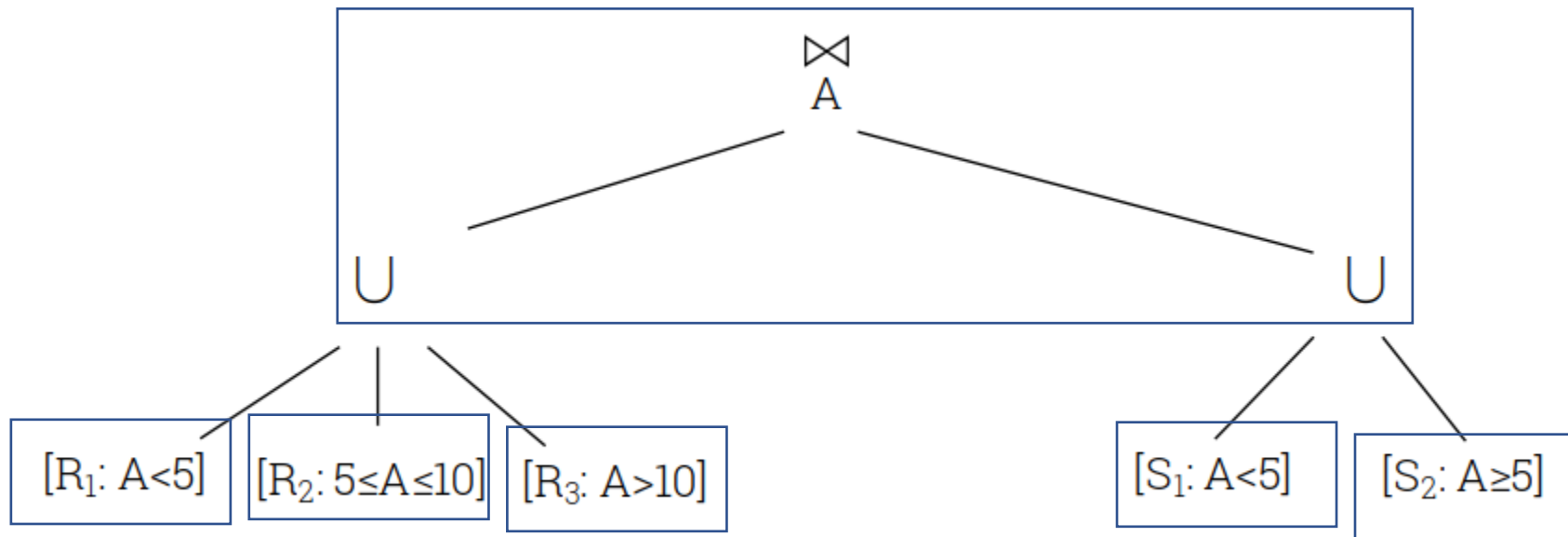
```
SELECT * FROM R JOIN S ON R.A = S.A
```



A = atributo en común

Localización

- **Ejemplo 2:**

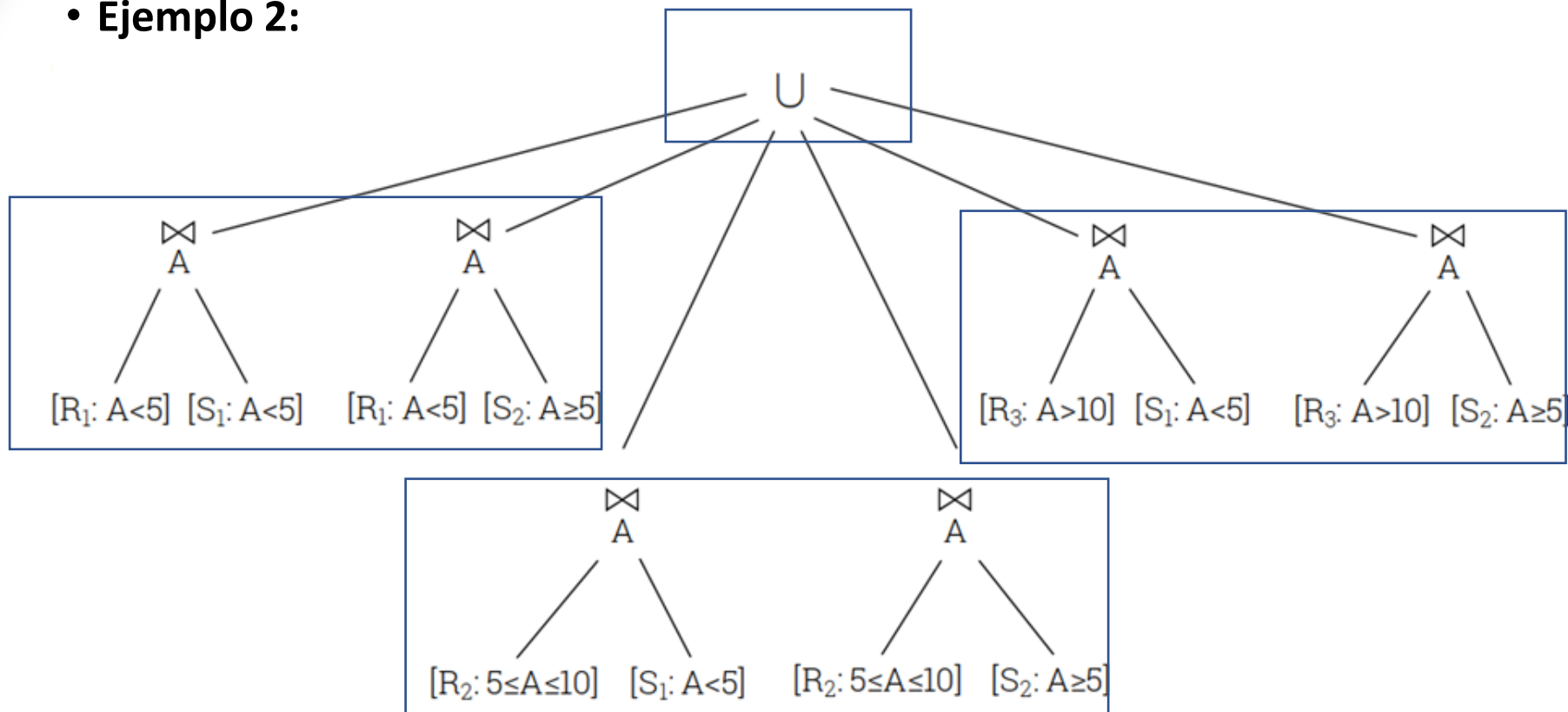


Fragmentación en R

Fragmentación en S

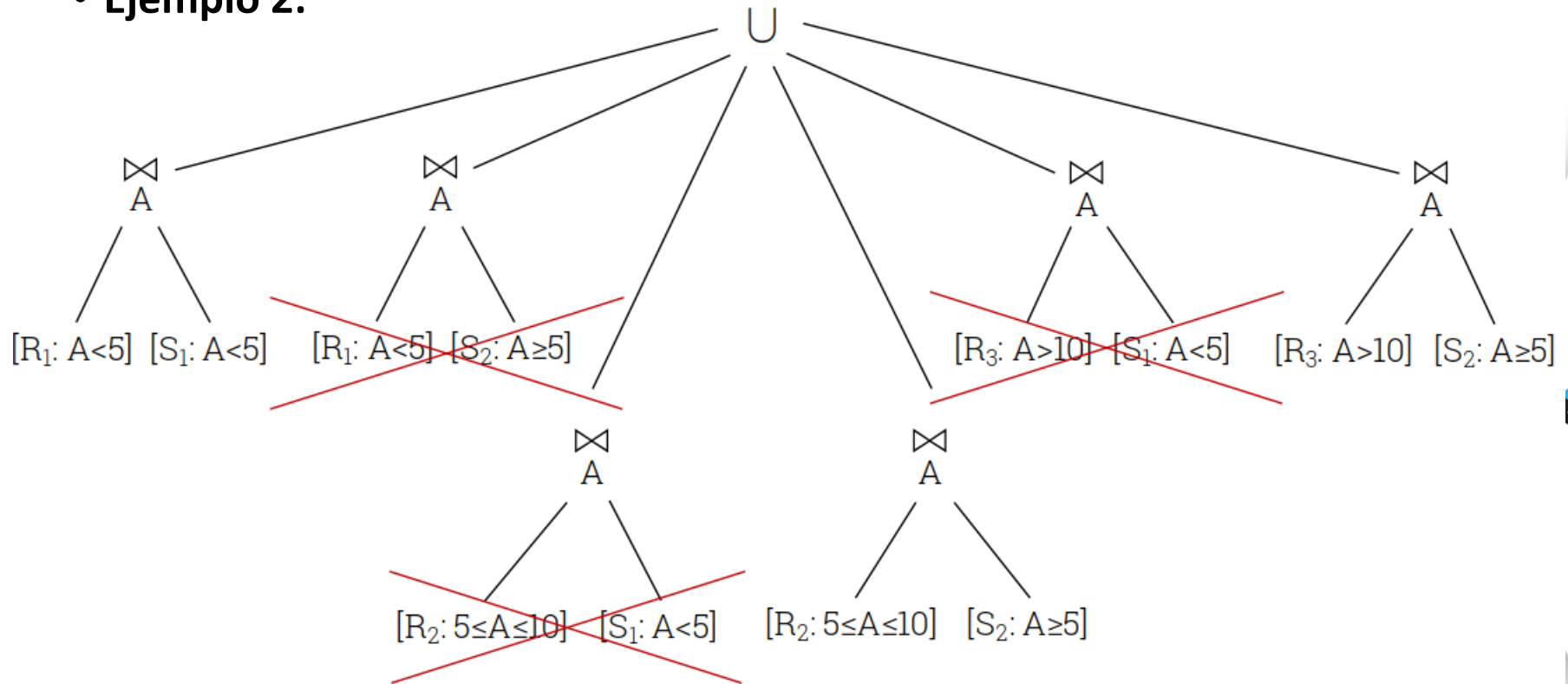
Localización

- **Ejemplo 2:**



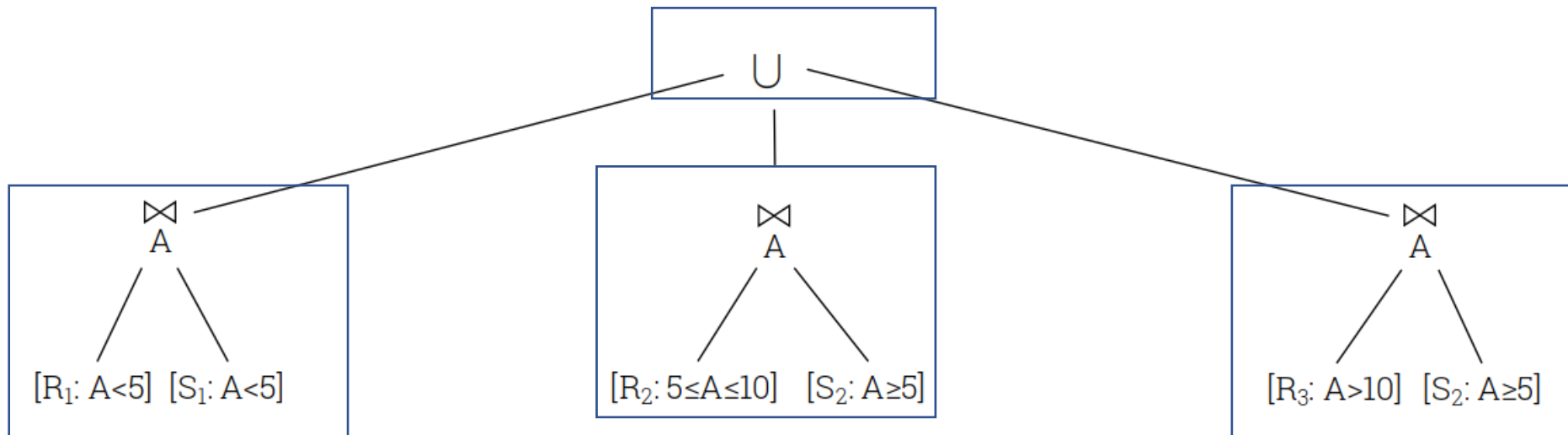
Localización

- **Ejemplo 2:**



Localización

- **Ejemplo 2:**



Localización

- **Regla 2:**

$$[R: c_1] \bowtie_A [S: c_2] \Rightarrow [R \bowtie_A S: c_1 \wedge c_2 \wedge R.A = S.A]$$

$$[R: \text{false}] \Rightarrow \emptyset$$

Localización

- **Ejemplo 2:**

$$[R_1: A < 5] \bowtie_A [S_2: A \geq 5]$$

$$\Rightarrow [R_1 \bowtie_A S_2: R_1.A < 5 \wedge S_2.A \geq 5 \wedge R_1.A = S_2.A]$$

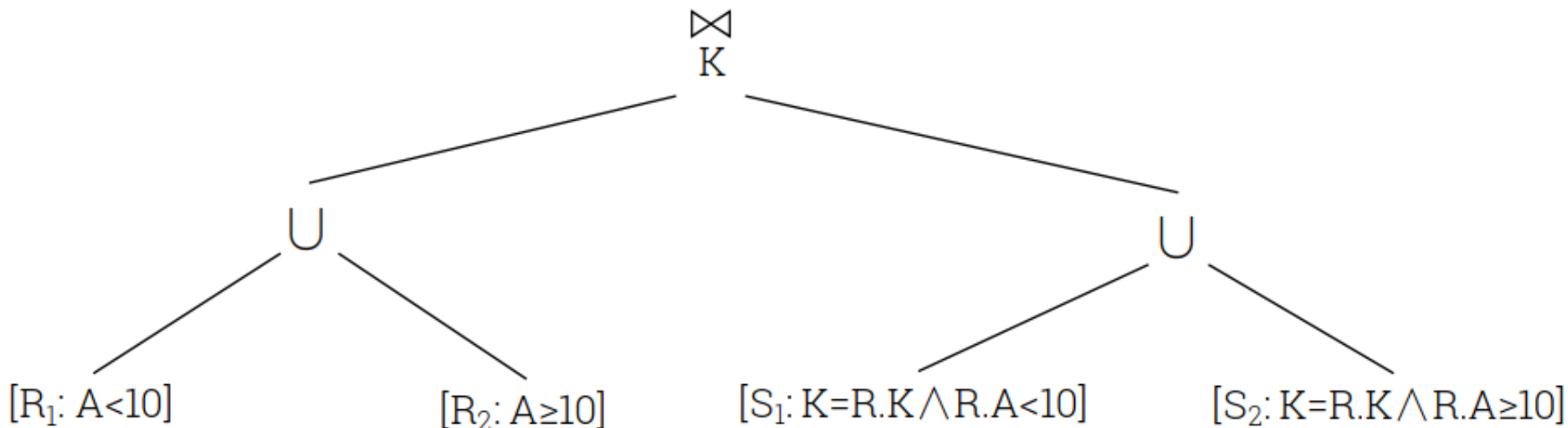
$$\Rightarrow [R_1 \bowtie_A S_2: \text{false}]$$

Localización

- **Ejemplo 3:**

Sea la consulta:

`SELECT * FROM R, S WHERE R.K = S.K`

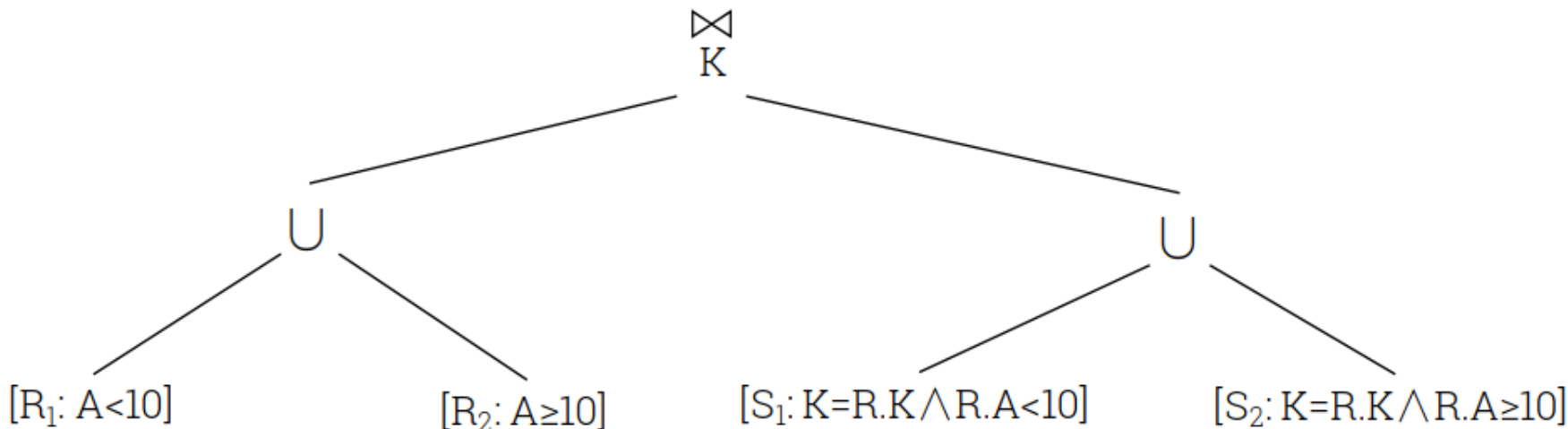


Localización

- **Ejemplo 3:**

Sea la consulta:

`SELECT * FROM R, S WHERE R.K = S.K`



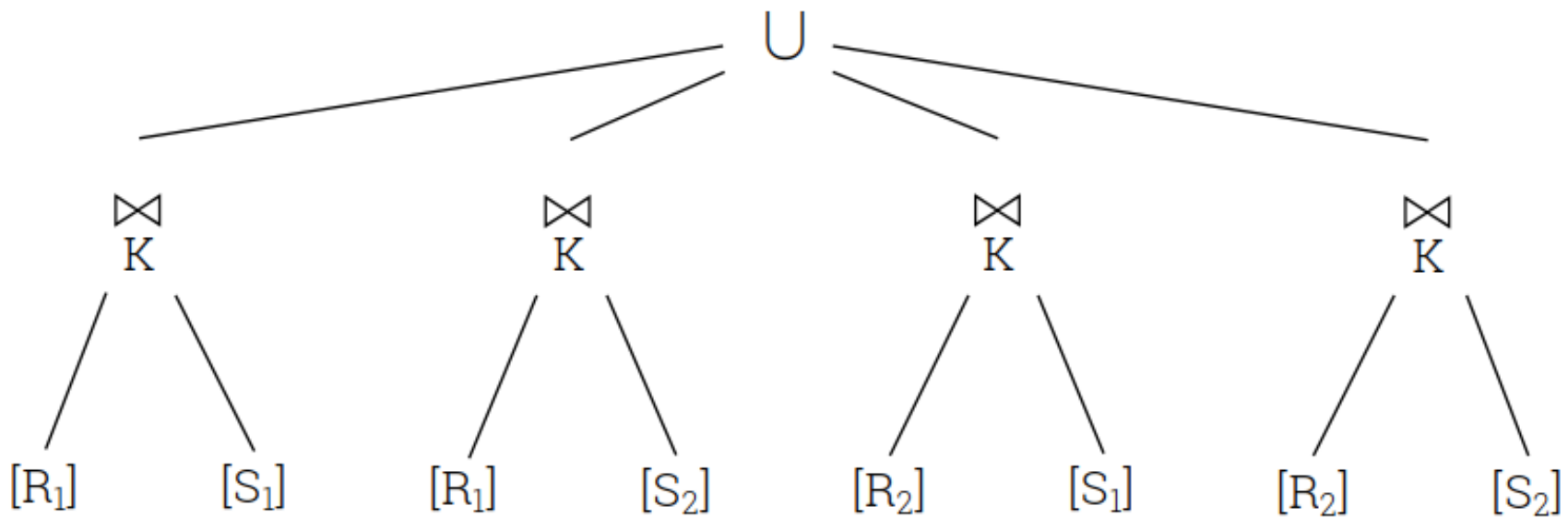
Fragmentación Derivada

$S_1 : S \text{ sj_k } R_1$

$S_2 : S \text{ sj_k } R_2$

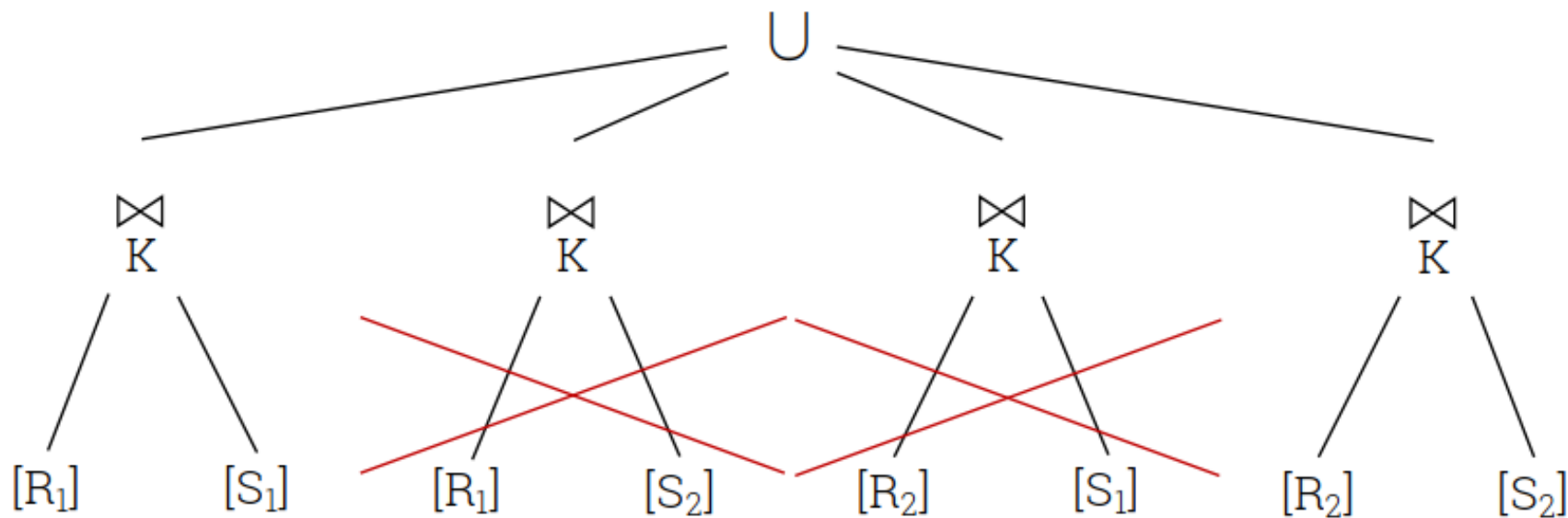
Localización

- **Ejemplo 3:**



Localización

- **Ejemplo 3:**



Localización

- **Ejemplo 3:**

$$[R_1: A < 10] \bowtie_K [S_2: K = R.K \wedge R.A \geq 10]$$

$$\Rightarrow [R_1 \bowtie_K S_2: R_1.A < 10 \wedge S_2.K = R.K \wedge R.A \geq 10 \wedge R_1.K = S_2.K]$$

$$\Rightarrow [R_1 \bowtie_K S_2: \text{false}]$$

$$\Rightarrow \emptyset$$

Localización

- **Ejemplo 4:**

- Sea la consulta:

SELECT A FROM R

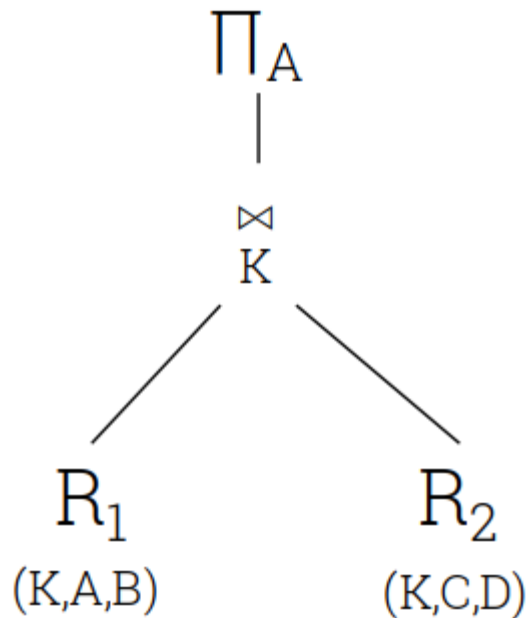
Y R esta compuesto por los fragmentos

{ R1(K,A, B) , R2(K,C,D)}

Localización

- Ejemplo 4:

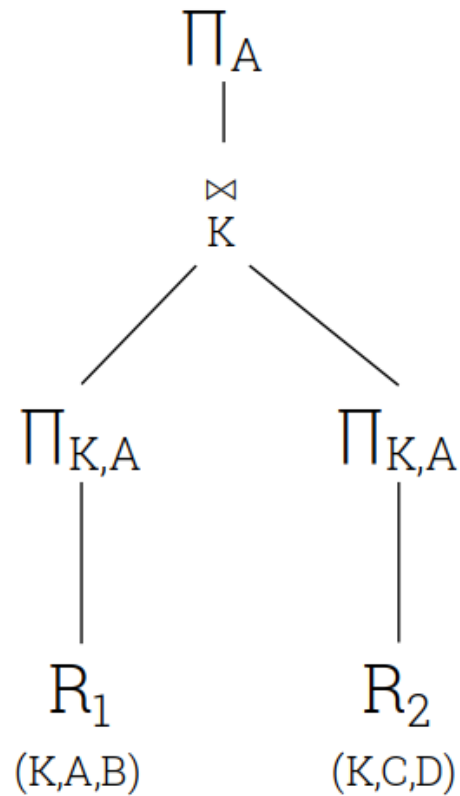
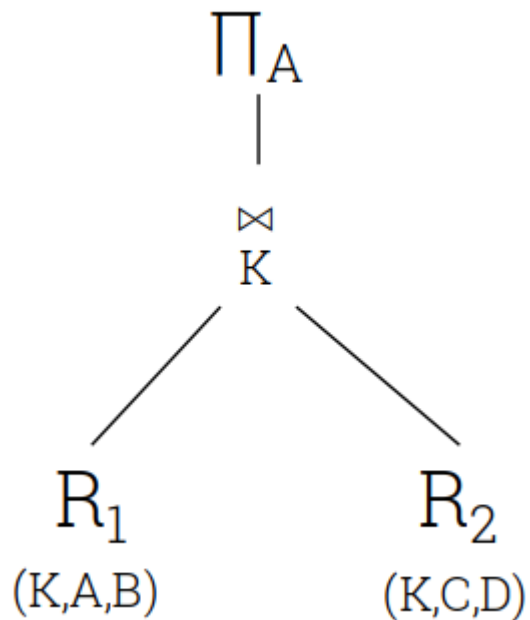
$$\begin{array}{c} \Pi_A \\ | \\ R \end{array}$$



Fragmentación Vertical

Localización

- **Ejemplo 4:**



Localización

- **Ejemplo 4:**

$$\begin{array}{c} \Pi_A \\ | \\ R_1 \\ (K, A, B) \end{array}$$

Localización

- **Regla 3:**

- Considerando la fragmentación vertical

$$R_i = \Pi_{A_i}(R), A_i \subseteq A$$

- Entonces, para algún $B \subseteq A$

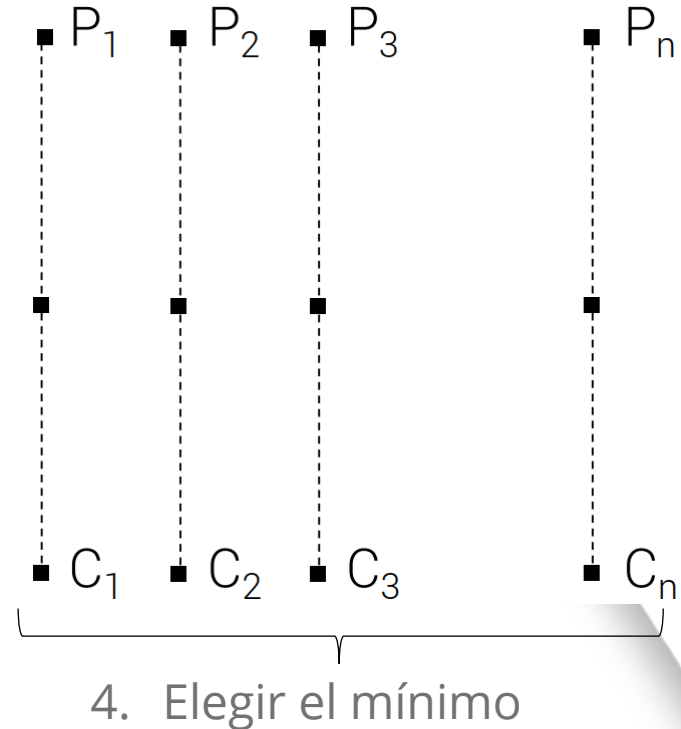
$$\Pi_B(R) = \Pi_B(\bowtie_i R_i \mid B \cap A_i \neq \emptyset)$$

Optimización

UTEC

Optimización: overview

- Proceso de optimización
 1. Generar planes de consulta.
 2. Estimar el tamaño de los resultados intermedios.
 3. Estimar el costo del plan



Optimización: overview

- Diferencias respecto a la optimización centralizada.
 - Nuevas estrategias para algunas operaciones.
 - Ordenación paralela / distribuida
 - Join paralelo / distribuido
 - Eliminación de duplicados
 - Agregación
 - Selección
- Muchas formas de asignar y programar procesadores.

Optimización: Ordenación paralela / distribuida

- **Casos de entrada:**

- a) La relación R en un solo sitio /disco
- b) La relación R fragmentado por el atributo de ordenación
- c) La relación R fragmentado por otro atributo

Optimización: Ordenación paralela / distribuida

- **Casos de salida:**

- a) La relación R ordenado en un solo sitio /disco
- b) Fragmentos ordenados de R

k	
5	...
6	
10	

F_1

k	
12	...
15	

F_2

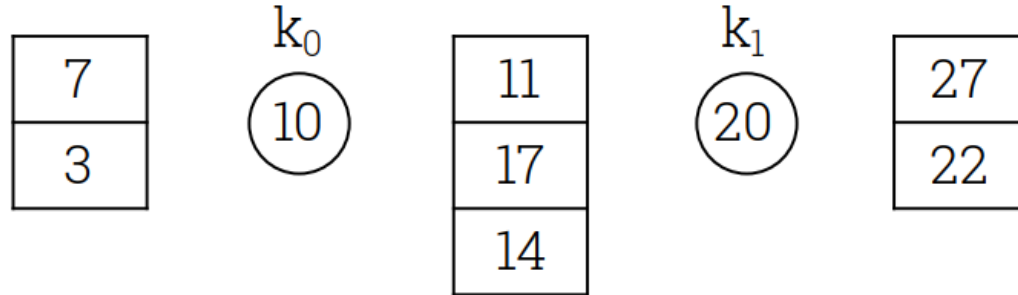
k	
19	...
20	
21	
50	

F_3

Optimización: Ordenación paralela / distribuida

- **Caso b) R fragmentado por el atributo de ordenación:**

- $R(k, \dots)$ para ser ordenado en k . `Select * from R order by k`
- R está fragmentado horizontalmente en k usando el vector $[k_0, k_1, \dots, k_n]$



Optimización: Ordenación paralela / distribuida

- **Caso b) R fragmentado por el atributo de ordenación:**
 - Algoritmo:
 1. Ordenar cada fragmento de forma independiente
 2. Enviar los resultados si es necesario

Optimización: Ordenación paralela / distribuida

- **Caso c) R fragmentado por otro atributo :**

- $R(k, \dots)$ para ser ordenado en k . `Select * from R order by k`
- R ubicado en uno o más sitios, **no** esta fragmentado en K

R_a

R_b

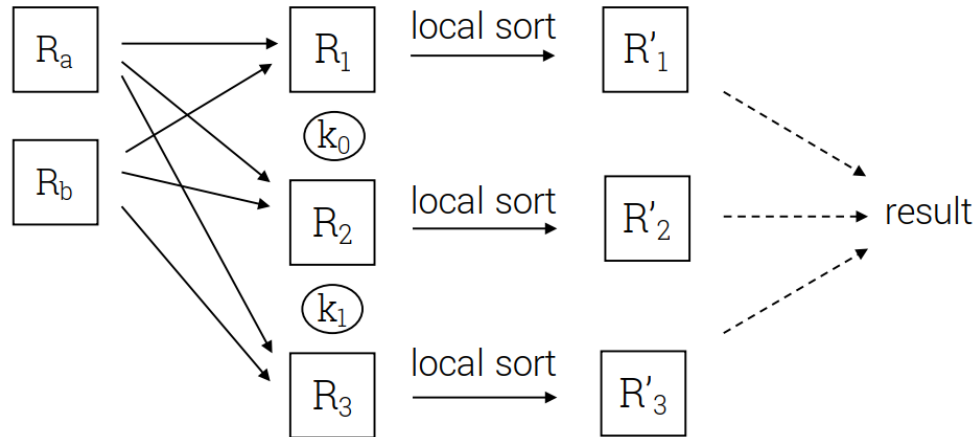
Optimización: Ordenación paralela / distribuida

- **Caso c) R fragmentado por otro atributo :**

Select * from R order by k

- Algoritmo

1. Fragmentación intermedia en k
2. Ordenación básica en cada fragmento intermedio



Optimización: Ordenación paralela / distribuida

- **Vector de partición:**

- Problema:

- Seleccione un buen vector de partición dado los fragmentos:

7	...
52	
11	
14	

R_a

31	...
8	
15	
11	
32	
17	

R_b

10	...
12	
4	

R_c

Optimización: Ordenación paralela / distribuida

- **Vector de partición:**

- Estrategía:

- Cada sitio envía al coordinador

- MIN valor de k
 - MAX valor de k
 - Número de tuplas

- Coordinador

- Calcula el vector y distribuye a los sitios.
 - Decide el número de sitios para realizar las ordenaciones locales.

Optimización: Ordenación paralela / distribuida

- **Vector de partición:**

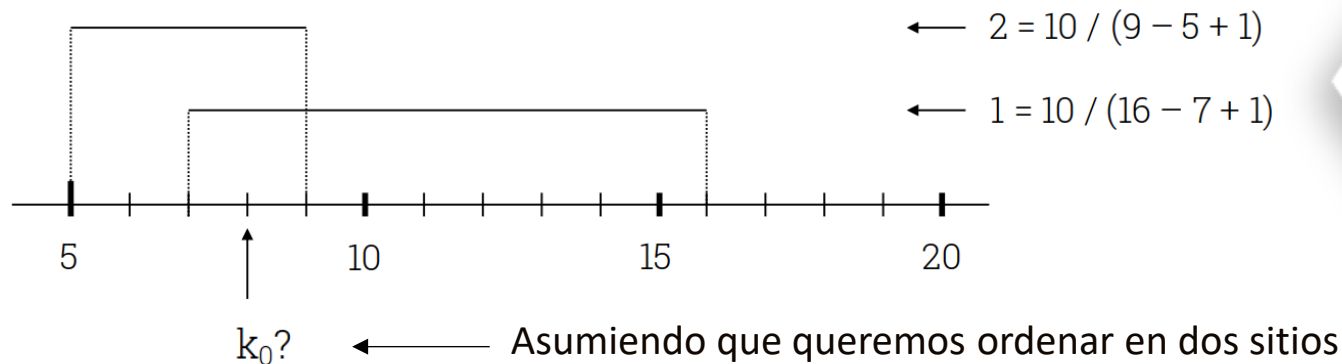
- Ejemplo:

- El coordinador recibe:

S_A : $\text{MIN} = 5$ $\text{MAX} = 9$ # tuplas = 10

S_B : $\text{MIN} = 7$ $\text{MAX} = 16$ # tuplas = 10

- Tuplas esperadas

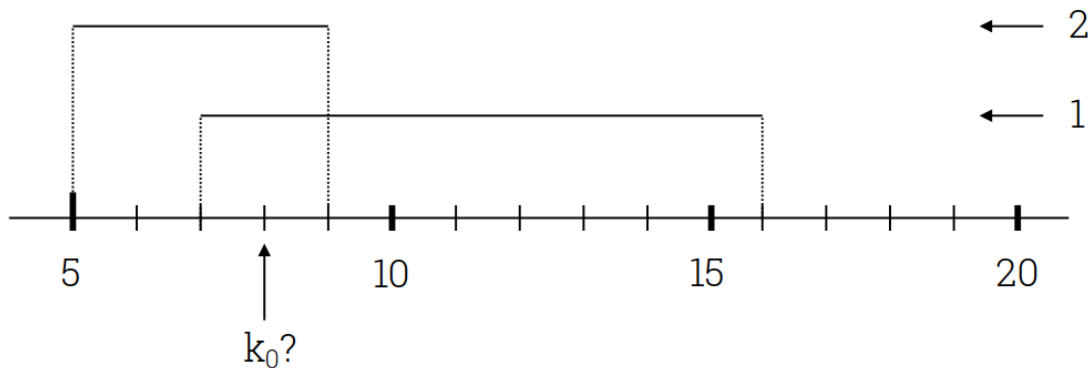


Optimización: Ordenación paralela / distribuida

- **Vector de partición:**

- Ejemplo:

- Tuplas esperadas



- Tuplas esperadas con clave $< k_0$ = la mitad de las tuplas totales

$$2(k_0 - 5) + (k_0 - 7) = 10$$
$$k_0 = (10 + 10 + 7) / 3 = 9$$

Optimización: Ordenación paralela / distribuida

- **Vector de partición:**

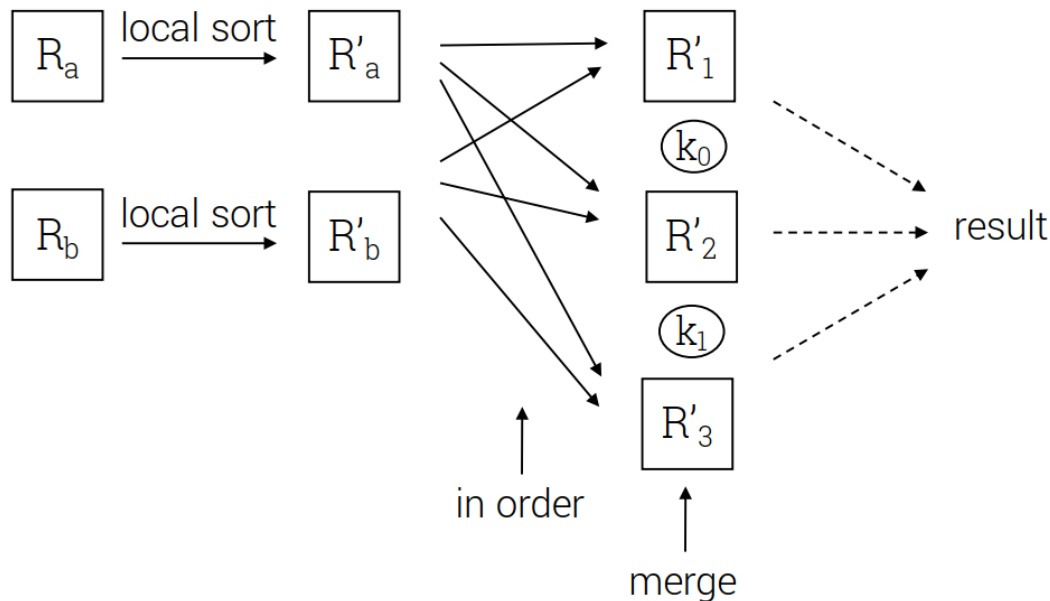
- Variantes

Herodotos Herodotou, Nedyalko Borisov, and Shivrath Babu. 2011. **Query optimization techniques for partitioned tables**. In Proceedings of the 2011 ACM SIGMOD International Conference on Management of data (SIGMOD '11). ACM, New York, NY, USA

Optimización: Ordenación paralela / distribuida

- **Ordenación usando Merge:**

- Primero se ordena localmente, y luego se mezclan en una fragmentación intermedia en K.



Optimización: Join paralelo / distribuido

- **Entrada:**

- Relaciones R y S
- Podrían o no estar fragmentados

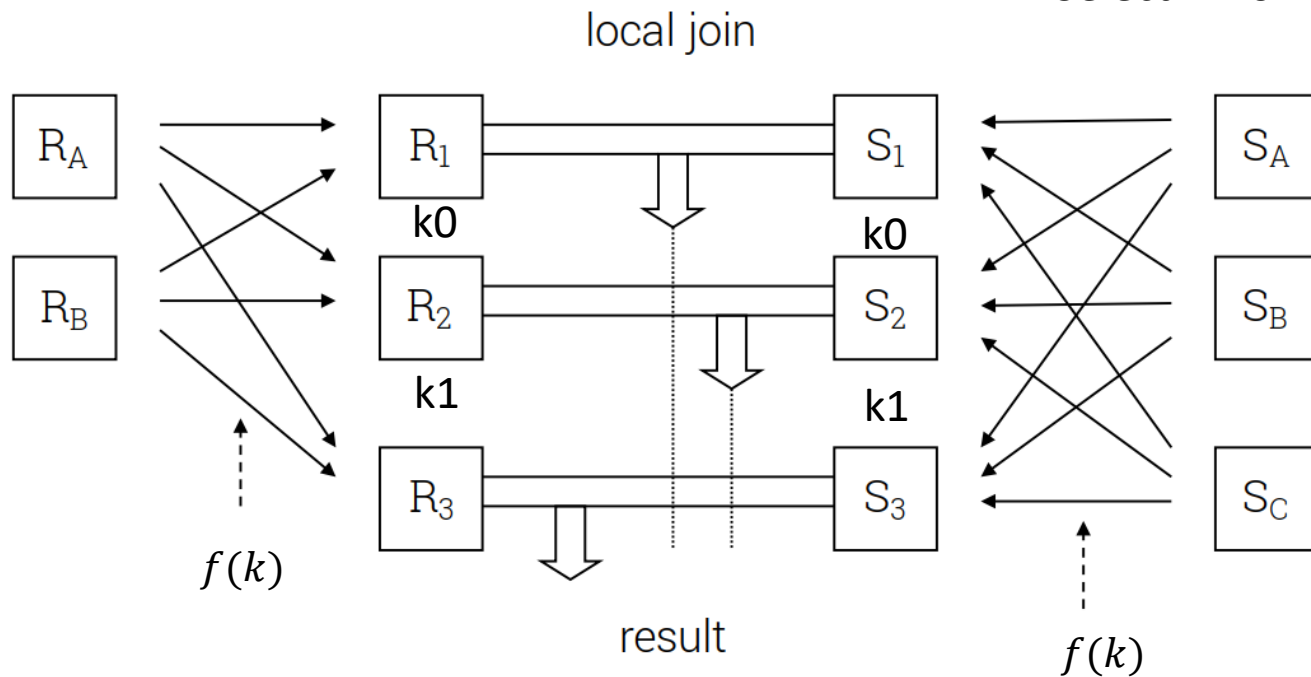
- **Salida:**

- $R \bowtie S$
- El resultado es alojado en uno o más sitios

Optimización: Join paralelo / distribuido

- **Join particionado (Equijoin):**

Select * from R join S on R.k = S.k



Optimización: Join paralelo / distribuido

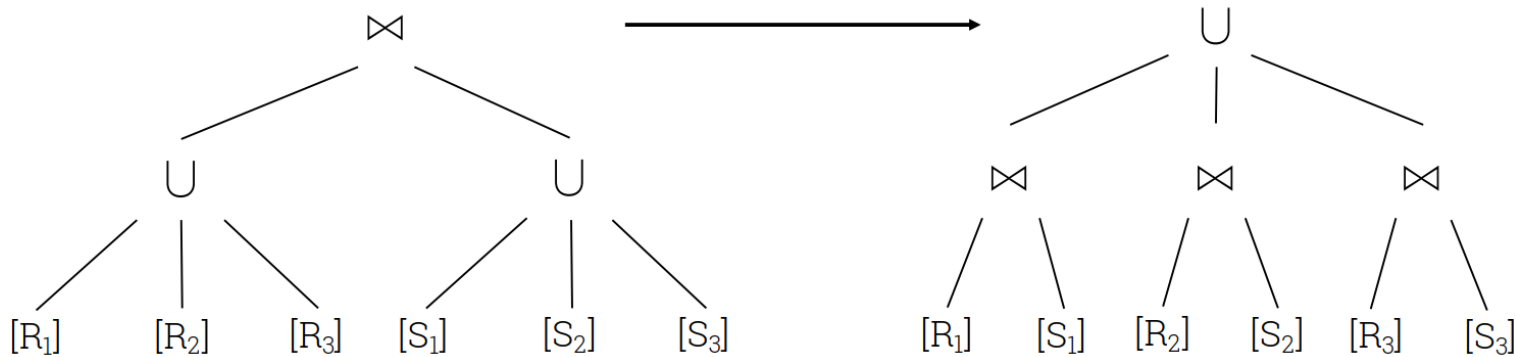
- **Join particionado (Equijoin):**

- Se utiliza la misma función de partición f para R y S
- f puede ser partición por rango o por hash
- El join local puede ser de cualquier tipo (cs2701)
- Varias opciones de programación, por ejemplo:
 - a) Particionar R; particionar S; ordenar y join
 - b) Particionar R; construir tablas hash locales para R; particionar S; join.

Optimización: Join paralelo / distribuido

- **Join particionado (Equijoin):**

- Ya sabemos por qué funciona.



- A veces particionamos solo para hacer posible este join

Optimización: Join paralelo / distribuido

- **Join particionado (Equijoin):**

- Seleccionar una buena función de partición f es muy importante.
- El objetivo es hacer todo $|R_i| + |S_i|$ el mismo tamaño.

Optimización: Otras operaciones paralelas

- **Eliminación de duplicados**

Select distinct k from R

- [Opción 1] Fragmentación intermedia en k (rango o hash), eliminar localmente o eliminar en el merge.
- [Opción 2] Ordenar primero (en paralelo) luego eliminar duplicados en el resultado.

- **Agregación**

- Partición por el atributo de agrupación.
- Calcular la función de agregación localmente.

Optimización: Agregación en Paralelo / Distribuido

R_a

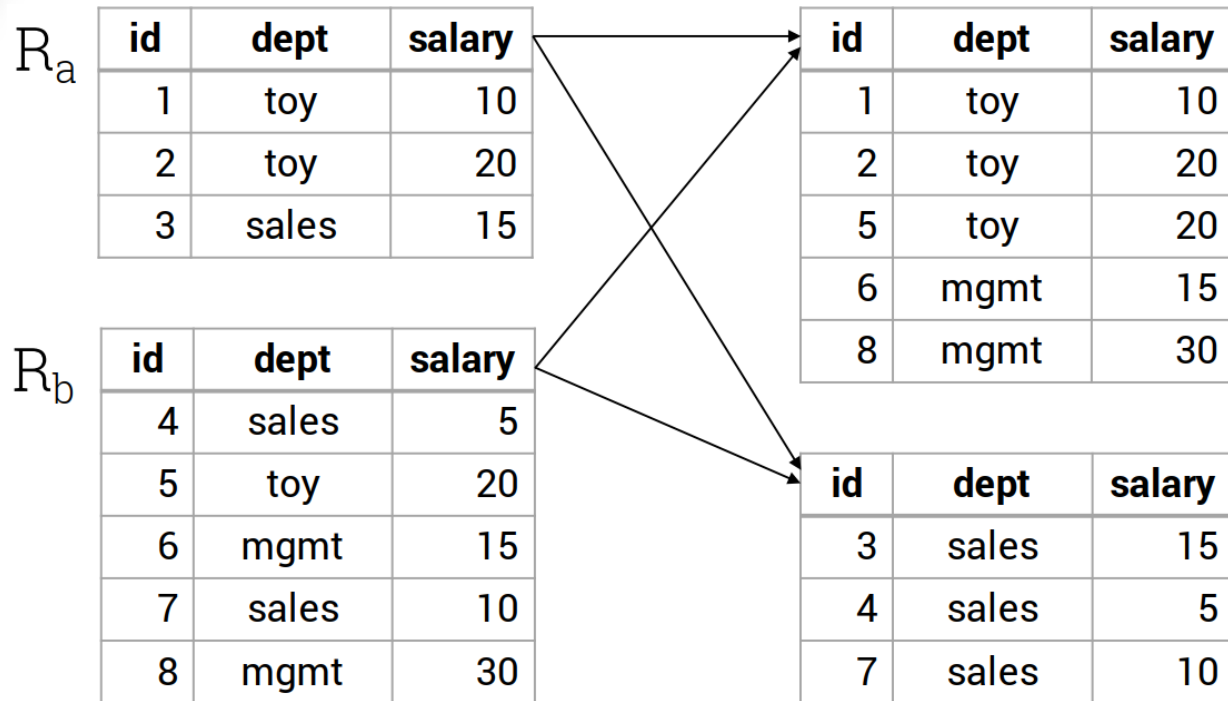
id	dept	salary
1	toy	10
2	toy	20
3	sales	15

R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

`select dept, sum (sal) from R group by dept`

Optimización: Agregación en Paralelo / Distribuido



`select dept, sum (sal) from R group by dept`

Optimización: Agregación en Paralelo / Distribuido

R_a

id	dept	salary
1	toy	10
2	toy	20
3	sales	15

id	dept	salary
1	toy	10
2	toy	20
5	toy	20
6	mgmt	15
8	mgmt	30

sum

dept	sum
toy	50
mgmt	45

R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

id	dept	salary
3	sales	15
4	sales	5
7	sales	10

sum

dept	sum
sales	30

`select dept, sum (sal) from R group by dept`

Optimización: Agregación en Paralelo / Distribuido

R_a

id	dept	salary
1	toy	10
2	toy	20
3	sales	15

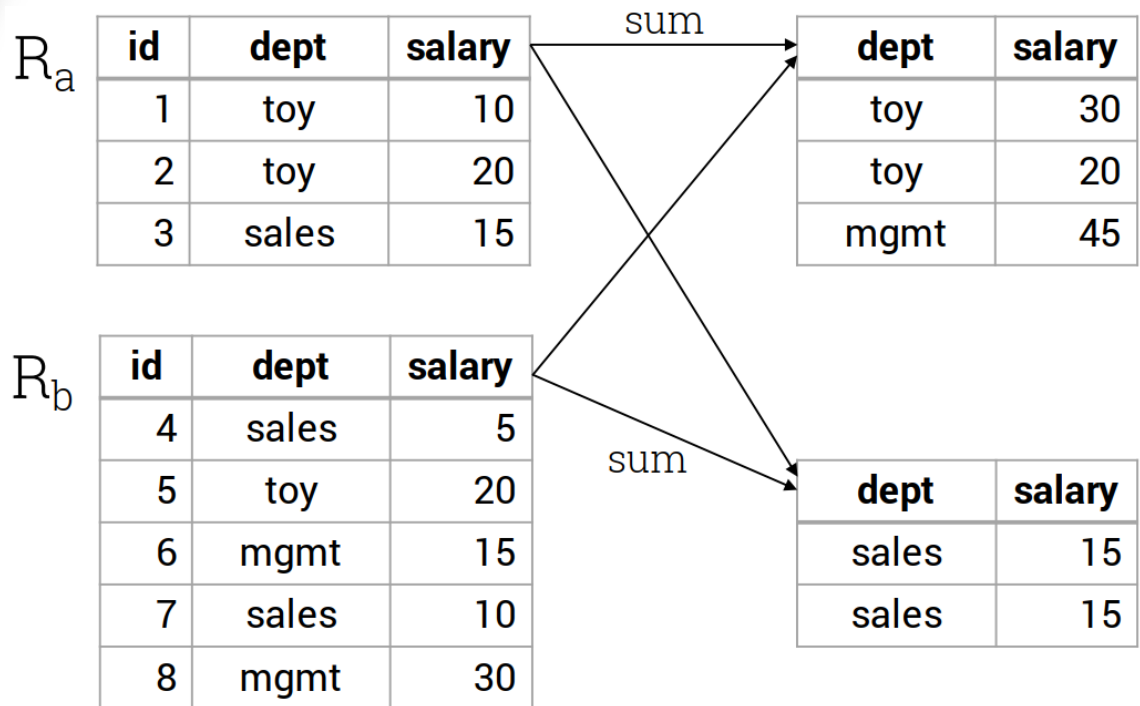
R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

select dept, sum (sal) from R group by dept

con menos datos

Optimización: Agregación en Paralelo / Distribuido



select dept, sum (sal) from R group by dept

con menos datos

Optimización: Agregación en Paralelo / Distribuido

con menos datos

R_a

id	dept	salary
1	toy	10
2	toy	20
3	sales	15

sum

dept	salary
toy	30
toy	20
mgmt	45

sum

dept	sum
toy	50
mgmt	45

R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

sum

dept	salary
sales	15
sales	15

sum

dept	sum
sales	30

`select dept, sum(sal) from R group by dept`

Optimización: Agregación en Paralelo / Distribuido

- **Mejoras:**

- Realizar la agregación durante la partición para reducir los datos transmitidos.
- → No funciona igual para todas las funciones agregadas,
 - ¿Cómo se optimiza la consulta para las funciones count, max, min, avg?

Optimización: Agregación en Paralelo / Distribuido

R_a

id	dept	salary
1	toy	10
2	toy	20
3	sales	15

R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

select dept, avg(sal) from R group by dept

Optimización: Agregación en Paralelo / Distribuido

Sum as S, count as C

Sum(S) / Sum(C)

R_a

id	dept	salary
1	toy	10
2	toy	20
3	sales	15

dept	s	c
toy	30	2
toy	20	1
mgmt	45	2

dept	avg
toy	50/3
mgmt	45/2

R_b

id	dept	salary
4	sales	5
5	toy	20
6	mgmt	15
7	sales	10
8	mgmt	30

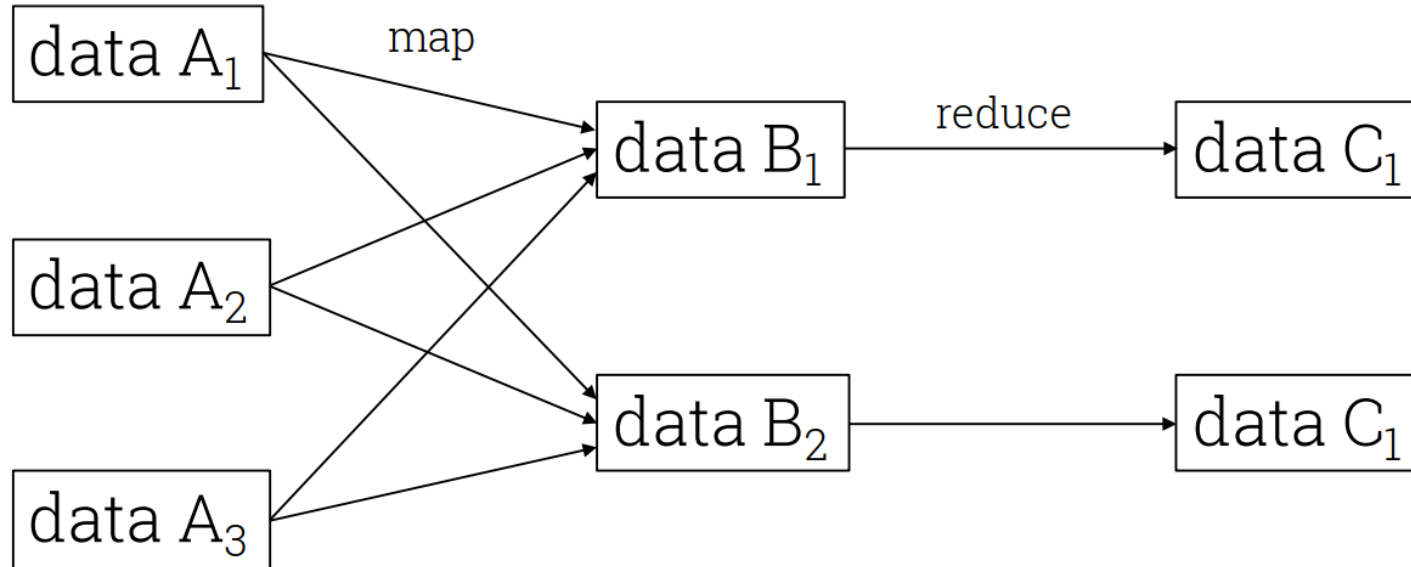
dept	s	c
sales	15	1
sales	15	2

dept	avg
sales	30/3

select dept, avg(sal) from R group by dept

Optimización: Agregación en Paralelo / Distribuido

- **MapReduce** (previa)

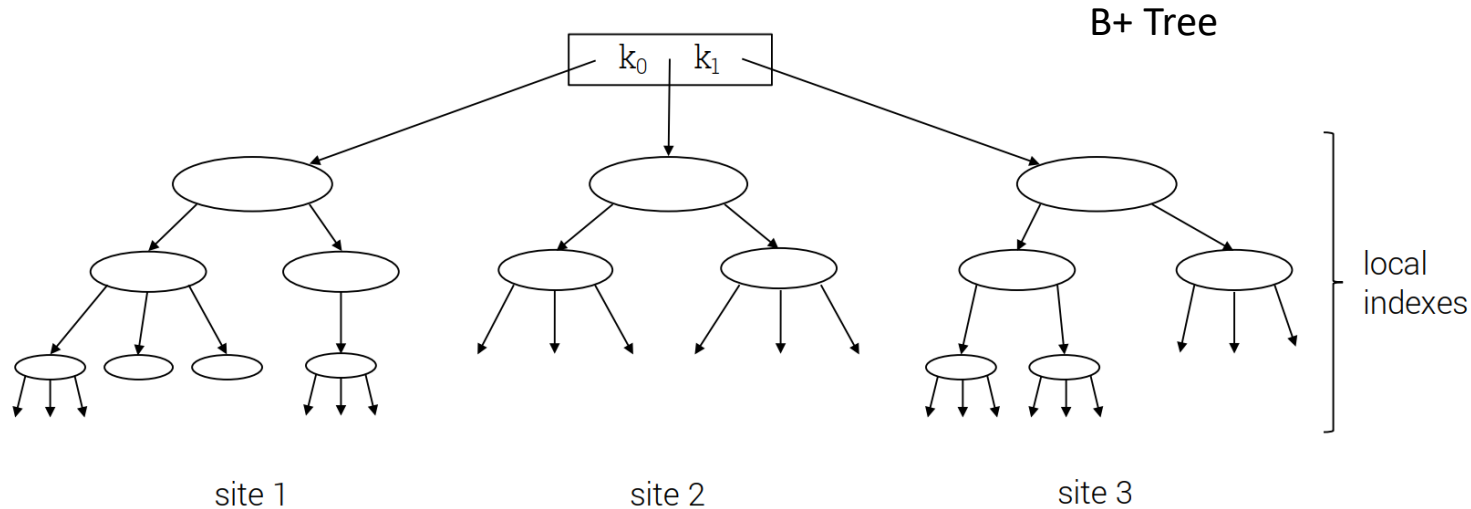


Optimización: Selección en Paralelo / Distribuido

- Sencillo si uno puede aprovechar el rango o la partición hash
- ¿Cómo funcionan los índices?

Optimización: Selección en Paralelo / Distribuido

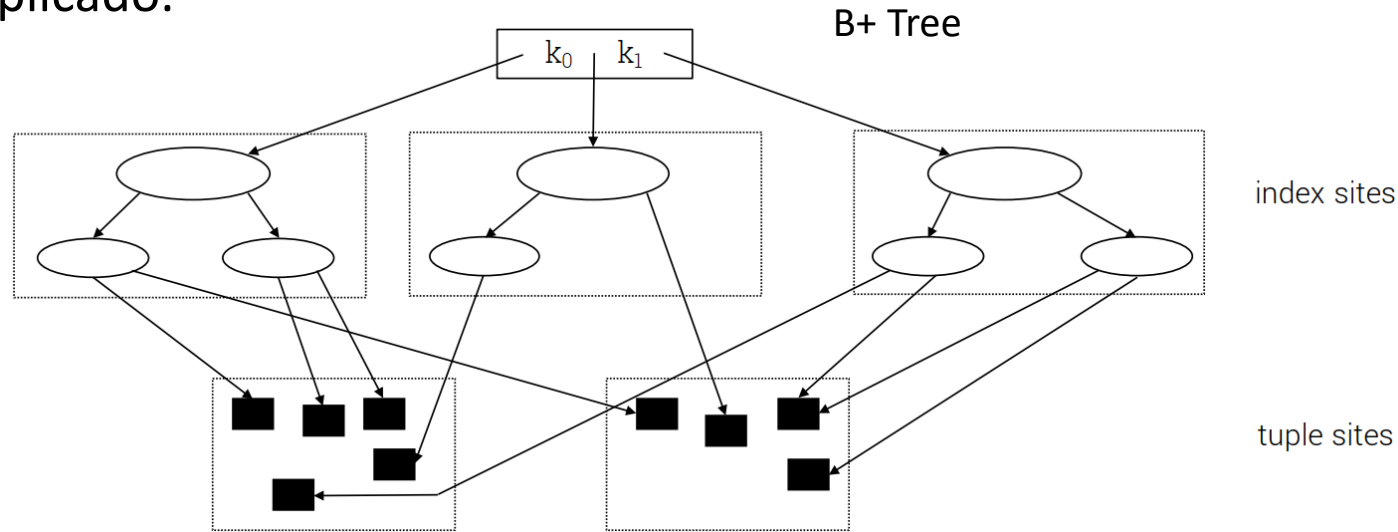
- El vector de partición puede actuar como la raíz de un índice distribuido:



Las tuplas de los nodos hojas están alojados en el mismo sitio.

Optimización: Selección en Paralelo / Distribuido

- Los índices distribuidos sobre atributos que no son de partición es complicado.



Las tuplas de los nodos hojas están alojados en diferentes sitios.

Optimización: Selección en Paralelo / Distribuido

- Esquemas de indexación
 - ¿Cuál es mejor en un entorno distribuido?
 - ¿Cómo hacer que las actualizaciones y la expansión sean eficientes?
 - ¿Dónde almacenar el directorio y el conjunto de participantes?
 - ¿Es necesario un conocimiento global?
- → Si el índice no es demasiado grande, puede ser mejor mantenerlo completo y replicarlo.

Optimización: overview

- Diferencias respecto a la optimización centralizada.
 - Nuevas estrategias para algunas operaciones.
 - ✓ Ordenación paralela / distribuida
 - ✓ Join paralelo / distribuido
 - ✓ Eliminación de duplicados
 - ✓ Agregación
 - ✓ Selección
- Muchas formas de asignar y programar procesadores.

Laboratorio 13

